

Data de Execução: 09/03/2025

Semana 5 - Aula 1

Tópico Principal: Linguagens de programação Back-End – Node.js

Subtítulo/Tema Específico: Introdução ao Node.js

Código da Aula: [SIS]ANO2-C2-B1-S5-A1

Objetivos da Aula:

- Identificar as características fundamentais do Node.js.
- Compreender o funcionamento do modelo de I/O assíncrono e do *Event Loop*.

Exposição Detalhada do Conteúdo

Referência do Slide: [Slide 6 - Ponto de partida]

- **Definição:** O Node.js é uma plataforma que permite executar código JavaScript fora do navegador, ou seja, no lado do servidor (back-end). Ele utiliza o motor V8 do Google para garantir alta performance.
- **Aprofundamento (Busca Externa):** O motor V8, escrito em C++, compila o JavaScript diretamente em código de máquina antes de executá-lo, o que elimina a necessidade de interpretação em tempo real e resulta em uma velocidade de execução próxima a linguagens compiladas. [Informação complementar via busca externa].

Referência do Slide: [Slide 7 - O que é Node.js?]

- **Definição:** É uma ferramenta que unifica o desenvolvimento, permitindo que o programador use JavaScript tanto no front-end quanto no back-end.
- **Aprofundamento (Busca Externa):** Essa característica é conhecida como "Full Stack JavaScript". Ela reduz a curva de aprendizado para equipes de desenvolvimento e facilita o compartilhamento de bibliotecas de validação de dados entre o cliente e o servidor. [Informação complementar via busca externa].

Referência do Slide: [Slide 8/9 - Características do Node.js]

- **Definição:**
 - **Assíncrono e não bloqueante:** Realiza múltiplas tarefas simultâneas sem travar a execução.
 - **Single-threaded:** Usa apenas uma trilha de execução principal, mas gerencia milhares de conexões via *Event Loop*.
 - **NPM:** O maior ecossistema de bibliotecas do mundo, facilitando a adição de funções prontas aos projetos.

Exemplo Prático: Imagine um sistema de chat em tempo real. Enquanto um usuário envia uma mensagem, o servidor Node.js não fica "travado" esperando o banco de dados confirmar a gravação; ele já começa a processar a mensagem de outro usuário imediatamente.

Fechamento da Aula

- **Resumo:**
 1. O Node.js executa JavaScript no servidor usando o motor V8.
 2. Seu diferencial é o modelo assíncrono que permite alta escalabilidade.
 3. O NPM é essencial para gerenciar dependências e acelerar o desenvolvimento.
- **Curadoria Digital:** [Curso de Node.js - O que é Node.js #01](#)
- **Bibliografia:**
 1. CANTELON, Mike et al. **Node.js em Ação**. Rio de Janeiro: Ciência Moderna, 2014.
 2. NODE.JS. **Documentação oficial**. Disponível em: <https://nodejs.org/pt>.

Semana 5 - Aula 2

Tópico Principal: Linguagens de programação Back-End – Node.js

Subtítulo/Tema Específico: Primeiros passos com Node.js

Código da Aula: [SIS]ANO2-C2-B1-S5-A2

Objetivos da Aula:

- Instalar e configurar o ambiente Node.js.
- Desenvolver um servidor web básico utilizando o módulo `http` e o framework Express.

Recursos Adicionais:

- Computador com acesso à internet para download.
- Editor de código (VS Code recomendado).

Exposição Detalhada do Conteúdo

Referência do Slide: [Slide 6/7 - Instalação]

- **Definição:** A instalação é feita pelo site oficial e inclui automaticamente o gerenciador de pacotes NPM. O comando `node -v` valida se a instalação foi bem-sucedida.
- **Aprofundamento (Busca Externa):** Recomenda-se o uso da versão **LTS (Long Term Support)** para ambientes de produção, pois oferece maior estabilidade e suporte a longo prazo em comparação com a versão "Current". [Informação complementar via busca externa].

Referência do Slide: [Slide 11/12 - Criando um servidor web]

- **Definição:** Através do módulo nativo `http`, o Node.js permite criar um servidor que "escuta" requisições em uma porta específica (ex: 3000) e envia respostas ao navegador.
- **Exemplo Prático:** Um pequeno servidor configurado para responder "Bem-vindo ao meu servidor Node.js!" ao ser acessado via `localhost:3000`.

Referência do Slide: [Slide 14/15 - Framework Express]

- **Definição:** O Express é uma biblioteca (framework) que simplifica a criação de rotas e APIs, tornando o código mais limpo e organizado que o módulo nativo.
- **Aprofundamento (Busca Externa):** O Express utiliza o conceito de *Middlewares*, que são funções que interceptam a requisição antes dela chegar ao destino final, permitindo realizar autenticação ou tratamento de dados. [Informação complementar via busca externa].

Fechamento da Aula

- **Resumo:**
 1. A instalação do Node.js habilita o comando `node` e o `npm` no terminal.
 2. É possível criar servidores nativos, mas frameworks como Express são o padrão de mercado.
 3. APIs construídas com Node.js devolvem dados geralmente no formato JSON para aplicativos móveis.
- **Curadoria Digital:** O que é Node.js? | #AluraMais.
- **Bibliografia:**
 1. TEIXEIRA, Caio. **Dominando JavaScript com jQuery e Node.js**. São Paulo: Casa do Código, 2014.
 2. EXPRESSJS. **Guia de consulta rápida**. Disponível em: <https://expressjs.com/>.

Semana 5 - Aula 3

Tópico Principal: Linguagens de programação Back-End – Node.js

Subtítulo/Tema Específico: Aplicações avançadas com Node.js

Código da Aula: [SIS]ANO2-C2-B1-S5-A3

Objetivos da Aula:

- Implementar persistência de dados com bancos de dados (MongoDB/MySQL).
- Compreender a comunicação em tempo real via WebSockets.

Recursos Adicionais:

- Computador com acesso à internet.
- Editor de código e roteiro de prática.

Exposição Detalhada do Conteúdo

Referência do Slide: [Slide 6 - Prática: Estruturação]

- **Definição:** Aplicações reais devem ser modulares, separando rotas, controladores e modelos para facilitar a manutenção.
- **Aprofundamento (Busca Externa):** O padrão **MVC (Model-View-Controller)** ou a **Arquitetura em Camadas** são essenciais para evitar que o código se torne um "espaguete" impossível de testar. [Informação complementar via busca externa].

Referência do Slide: [Slide 6 - Persistência e WebSockets]

- **Definição:** * **Bancos de Dados:** Uso de ORMs (como Mongoose) para salvar informações de forma permanente.
 - **WebSockets (Socket.io):** Permite que o servidor envie dados para o cliente sem que ele peça, ideal para notificações instantâneas.
- **Exemplo Prático:** Um aplicativo de entrega onde o cliente vê a posição do entregador no mapa se movendo em tempo real, sem precisar atualizar a página (via WebSockets).

Fechamento da Aula

- **Resumo:**

1. Boas práticas incluem modularização e uso de middlewares para segurança.
2. O Node.js integra-se facilmente com bancos NoSQL como MongoDB.
3. A escalabilidade pode ser gerenciada com ferramentas de produção como PM2.

- **Curadoria Digital:** [Introdução ao Socket.io](#) (Tutorial sobre comunicação em tempo real).

- **Bibliografia:**

1. MARIANO, J. **Node.js: Aplicações de alta performance com JavaScript**. São Paulo: Novatec, 2018.
2. W3SCHOOLS. **Node.js Introduction**. Disponível em: <https://www.w3schools.com/nodejs/>.